

Databases

Lecture 9 - Trigger, Integrity and Security

Emily Lucia Antosch

HAW Hamburg

04.04.2026

Contents

1. Introduction	6
2. Repetition	10
Updating Views	11
Basics	12
Basics	13
Generated Tables	14
Generated Tables	15
Operations	16
ACID	17
ACID - Atomicity	18
ACID - Consistency	19
ACID - Isolation	20

ACID - Isolation: Concurrency Control	21
ACID - Isolation: Concurrency Control	22
ACID - Isolation: Concurrency Control	23
ACID - Isolation: Concurrency Control	24
ACID - Durability	25
ACID - Durability: Error Handling	26
ACID - Durability: Error Handling	27
ACID - Durability: Error Handling	28
ACID - Durability: Error Handling	29
ACID - Durability: Error Handling	30
ACID - Durability: Error Handling	31
Distributed Transactions	32
Distributed Transactions	33

Savepoints	34
Savepoints	35
3. Integrity, Trigger and Security	36
Integrity Constraints	37
Integrity Constraints	38
Integrity Constraints - ECA	39
Trigger Syntax	40
Excursion Delimiter	41
Excursion Delimiter	42
Excursion Delimiter: Example	43
Events	44
Types	45
Order of Trigger	47

Transition Variables	48
Use Cases	49
Trigger: Example	50
Trigger: Example	51
Problems	52
Problems	53
Assignment: Webshop	54
Assignment: Webshop	55
Permissions	56
GRANT and REVOKE	57
Least Privilege Principle	58
Assignment: Webshop	59
4. License Notice	60

1. Introduction

1.1 Where are we right now?

- Last time, we looked at the basics of subqueries and views
- Today, we'll be discussing
 - ▶ how we can use transactions to increase the safety of our data manipulation statements,
 - ▶ how transactions are executed and
 - ▶ what triggers are and how we can use them.

1.1 Where are we right now?

1. Introduction
2. Basics
3. SQL
4. Entity-Relationship-Model
5. Relationships
6. Constraints
8. **Subqueries & Views**
9. **Transactions**
10. Database Applications
11. **Integrity, Trigger & Security**

1.2 What is the goal of this chapter?

- At the end of this lesson, you should be able to
 - ▶ create and define functions being run certain events using triggers
 - ▶ use transactions to make safe changes, that can be undone if necessary.

2. Repetition

Updating Views

- Views are Relations, just like tables!
- Should make no difference to users

? Question

Can we modify the view's data?

- Depends on type of view!

Basics

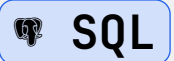
- Classify views based on the select:
 - ▶ **Projection View**
 - `SELECT a, b, c ...`
 - ▶ **Selection View**
 - `... WHERE <condition> ...`
 - ▶ **Join View**
 - `FROM tab_a JOIN tab_b ...`
 - **Aggregation View**
 - `SELECT MAX(x) ...`
- Other types and combinations exist

Basics

- A view with a single defining table is updatable if
 - ▶ the view attributes contain the primary key of the base relation,
 - ▶ as well as all attributes with the NOT NULL constraint that have a default value specified
- Views defined on multiple tables using joins are only updatable in special cases
 - ▶ E.g., INSERT and UPDATE for Join Views, if join condition is based on PK-FK
- Views defined using grouping and aggregate functions are not updatable

Generated Tables

```
1 CREATE TABLE <name> AS SELECT
```



- Can create new table based on query
- New table is independent from old table
- Use cases:
 - ▶ Copy table
 - ▶ Copy parts of table

! Memorize

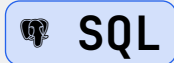
New table does not have all constraints of the parent table!

Generated Tables



Example

```
1 INSERT INTO Underpaid ( lname , fname )
2 SELECT lname , fname **FROM** Employee WHERE salary < 1000 ;
```



- WHERE clause belongs to SELECT

Operations

- A transaction bundles several operations into one logical unit
 - ▶ Unit of Work
- Includes one or more database access operations E.g., INSERT, DELETE, UPDATE, SELECT
- Operations must be executed all or none
- Example: Order a hotel room over the internet
 - ▶ Choose and reserve room
 - ▶ Payment
 - ▶ Final booking of the hotel room

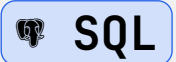
ACID

- Key features of transactions
 - ▶ **Atomicity**: Transaction is executed in whole or not at all
 - ▶ **Consistency**: State of the DB is consistent before and after a transaction
 - ▶ **Isolation**: Transactions do not interfere with other concurrent transactions
 - ▶ **Durability**: Changes are stored permanently in the database and will not get lost

ACID - Atomicity

- Begin of Transaction (**BoT**)
 - ▶ SQL99: START TRANSACTION
 - ▶ PostgreSQL:

```
1  BEGIN;
```



- Commit a transaction: COMMIT;
 - ▶ All operations are made persistent
 - ▶ All changes are visible to other users
- Rollback transaction: ROLLBACK;
 - ▶ DB is in state at **BoT** again

ACID - Consistency

- DB: in consistent state before transaction Also, in consistent state after transaction
- Integrity constraints assure that
- Constraints can be defined as
 - ▶ IMMEDIATE (default in MySQL)
 - are checked immediately after operation
 - ▶ DEFERRED
 - Check at time of commit

ACID - Isolation

- Transactions are isolated from other concurrent transactions
- Concurrent transactions shall behave well

ACID - Isolation: Concurrency Control

- Concurrent operations can lead to problems
 - ▶ Lost Update
 - ▶ Dirty Read
 - Unrepeatable read
 - Phantom tuples

ACID - Isolation: Concurrency Control

- Lost Update is prevented by SQL
- Transactions: may choose **Isolation Level**
 - ▶ SERIALIZABLE
 - no problems
 - REPEATABLE READ (default in MySQL)
 - Open for phantom tuples
 - READ COMMITTED (default in Oracle, SQL Server)
 - Open for phantom tuples and unrepeatable read
 - READ UNCOMMITTED
 - Open for all problems

ACID - Isolation: Concurrency Control

TRANSACTION ISOLATION LEVELS
explained as if you were building a snowman

if you have more than one process trying to read and/or modify resource - you have concurrency. Isolation Levels dictate what happens in such scenarios.



READ UNCOMMITTED (Isolation Level: 1 icon)
Let's build a snowman together! Woohoo!

READ COMMITTED (Isolation Level: 2 icons)
I'll build a snowman in my backyard but you can come and see it occasionally m'kay

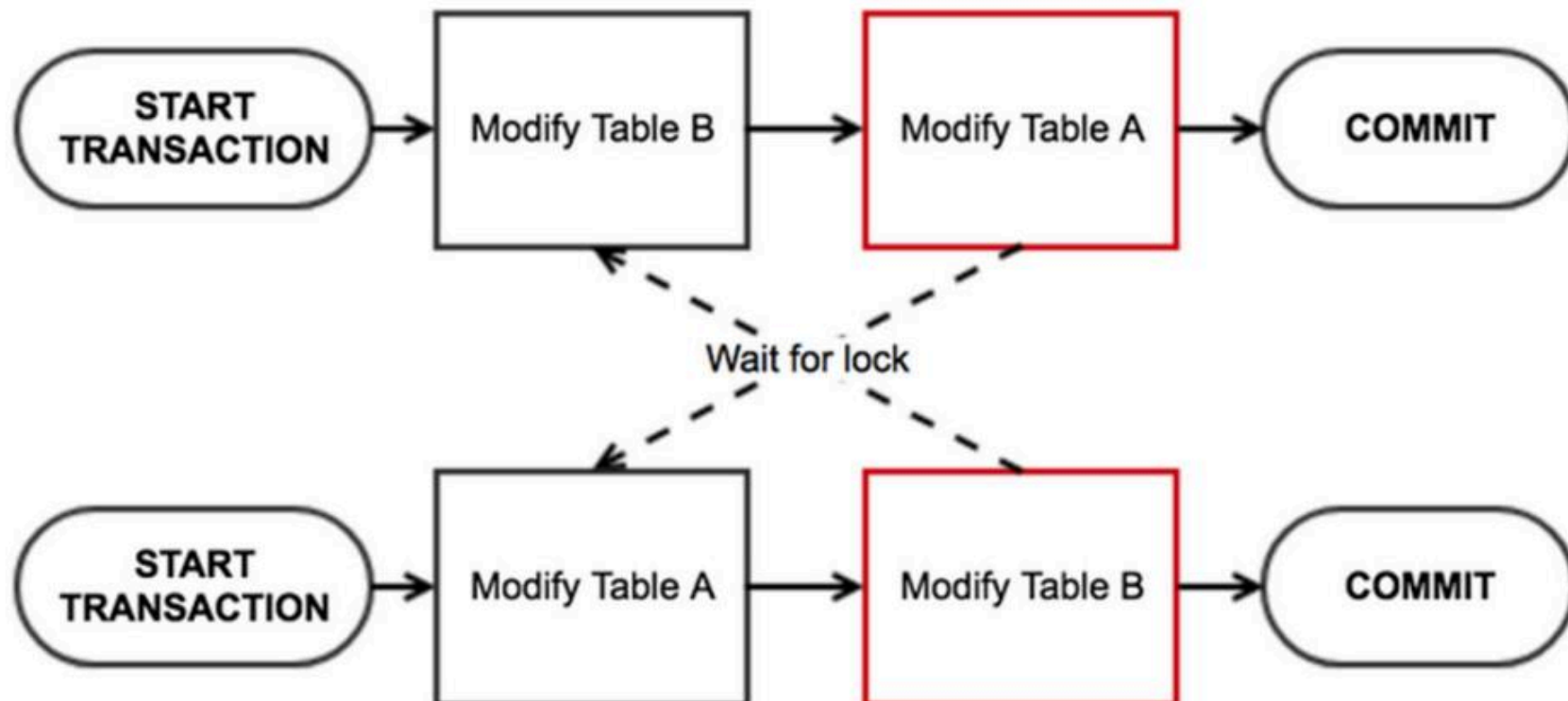
REPEATABLE READ (Isolation Level: 3 icons)
I'll just show you a picture of my snowman but you won't see it until I'm done that's fair

SERIALIZABLE (Isolation Level: 4 icons)
You won't even know what I'm building until I'm done! that sucks!

BitesizedEngineering.com

ACID - Isolation: Concurrency Control

- Deadlocks may occur!
 - ▶ Usually are resolved automatically by aborting one transaction



ACID - Durability

- Once committed, changed data is safe
- Error types
 1. Computer failure
 2. Transaction or system error (constraint violation, $\frac{x}{0}$, blackout, system crash)
 3. Local Errors
 4. Concurrency control enforcement
 5. Disk error (harddisk broken)
 6. Physical problems and catastrophes (fire, earthquake, robbery, ...)

ACID - Durability: Error Handling

- Recovery from transaction failures usually means that the database is **restored** to the most recent consistent state just before the time of failure
- Minor damages due to error types 1-4 from slide “ACID – Durability”
 - ▶ DBMS provides handling
 - Recovery strategy is to identify any changes that may cause an inconsistency in the database
 - Changes are first written to redo logs (files on disk)
 - Written to database files after commit

ACID - Durability: Error Handling

- Extensive damage due to error types 5-6 from slide “ACID – Durability”
 - ▶ Recovery handling restores a past copy of the database from archival storage
 - ▶ Reconstructs a more current state by redoing the operations
 - ▶ Last transactions are lost!
- Solution: Redundancy
 - ▶ RAID (**r** edundant **a** rray of **i** ndependent **d** isks)
 - Data Replication by DBMS

ACID - Durability: Error Handling

- Changes are performed on (replicated to) several database instances
- Master/Slave
 - ▶ Updates only on one instance (master)
 - ▶ Slave: Read only vs. Standby
- Multi-Master
 - ▶ Updates on different instances
 - ▶ Needs conflict resolution strategy

ACID - Durability: Error Handling

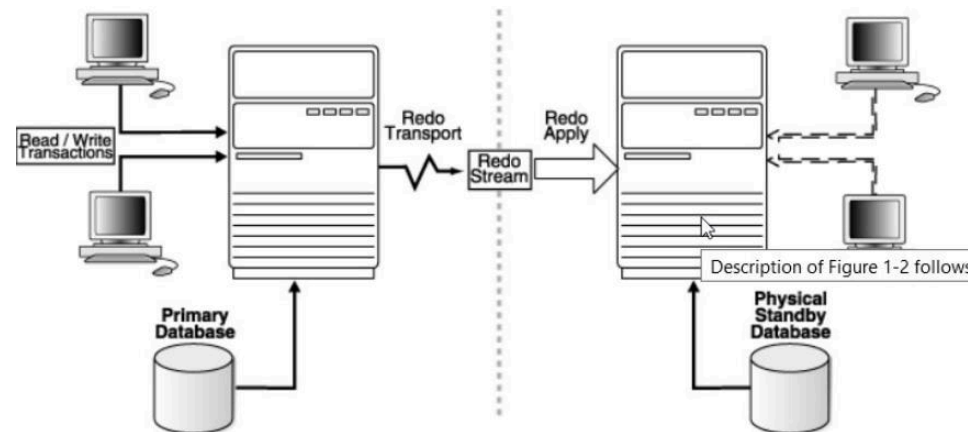
- **Synchronous**
 - ▶ Transaction valid only when committed on all DBs
 - ▶ Safest, but performance impact
 - ▶ May reduce availability of the system
- **Asynchronous**
 - ▶ Transaction valid when committed locally

ACID - Durability: Error Handling

- Low level (disk device)
- Trigger based
 - ▶ Update triggers the replication (SQL level)
- Logfile shipping
 - ▶ Changes are stored in redo logs (as usual)
 - ▶ redo logs are copied to standby DB

ACID - Durability: Error Handling

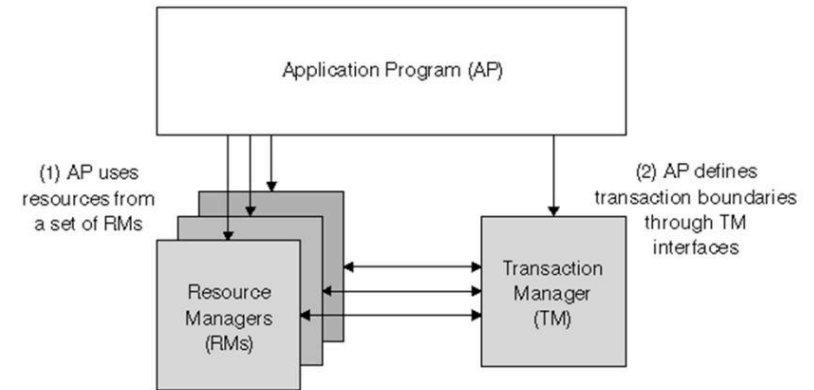
- Oracle
 - ▶ Data Guard
 - Replication on second server, can be used to answer Read-Only queries



- Real Application Cluster (RAC)
 - ▶ Several servers share the same DB

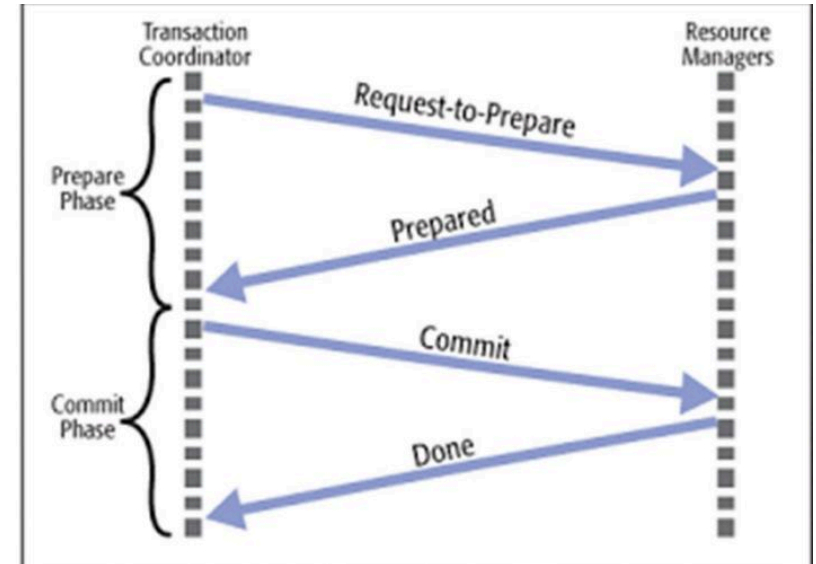
Distributed Transactions

- Transactions not only in a single DBS
- Standardized by X/Open
 - ▶ Transaction Manager: A software component that guarantees transaction properties
 - ▶ Resource Manager: Every resource (e.g., DBS, GUI) that is able to work in a transactional mode without providing a transaction control structure itself
- The Transaction manager coordinates the Resource Manager that take part in the transaction. E.g., different DBS (distributed transactions) that appear as one DBS from outside (transparency!)



Distributed Transactions

- To ensure interoperability between the participating resource managers the **2-phase commit protocol** is realized
- It defines the final synchronization of different parts of a transaction of a global transaction
- In the first phase the transaction manager asks participating resource managers to announce the results of their local transaction part
- This leads to a global result (commit or rollback) that is then in the second phase announced to the participants



Savepoints

- There are operations that may be expensive to execute time consuming
- If certain constraints fail within transaction execution, then maybe these constraints may not fail in a second attempt (e.g., time dependent)
- So “fall back” points can be defined, which are called **savepoints**
- It is possible to rollback up to a savepoint and restart transaction execution from this point on

Savepoints

Code

```
UPDATE STUDENT SET STUDENT_NAME = 'Mathew' WHERE STUDENT_NAME = 'Mahtwe';  
SAVEPOINT S1;  
UPDATE STUDENT SET AGE = 15 WHERE STUDENT_ID = 100;  
ROLLBACK to S1;
```

STUDENT			
STUDENT_ID	STUDENT_NAME	Address	Age
100	Joseph	Troy	22
101	Mahtwe	Lakeside Village	23
102	Jacob	Fraser Town	22

STUDENT			
STUDENT_ID	STUDENT_NAME	Address	Age
100	Joseph	Troy	22
101	Mathew	Lakeside Village	23
102	Jacob	Fraser Town	22



STUDENT			
STUDENT_ID	STUDENT_NAME	Address	Age
100	Joseph	Troy	15
101	Mathew	Lakeside Village	23
102	Jacob	Fraser Town	22

STUDENT			
STUDENT	STUDENT	Address	Age
100	Joseph	Troy	22
101	Mathew	Lakeside Village	23
102	Jacob	Fraser Town	22



3. Integrity, Trigger and Security

Integrity Constraints

- Static Constraints
 - ▶ Conditions on states
 - ▶ Conditions must be fulfilled before and after operations
 - ▶ Used until now
 - Primary Key
 - Foreign Key
 - UNIQUE, NOT NULL, CHECK
- Dynamic Constraints (**Assertions**)
 - ▶ Integrity conditions that affect multiple tables
 - ▶ Conditions on state transitions



Example

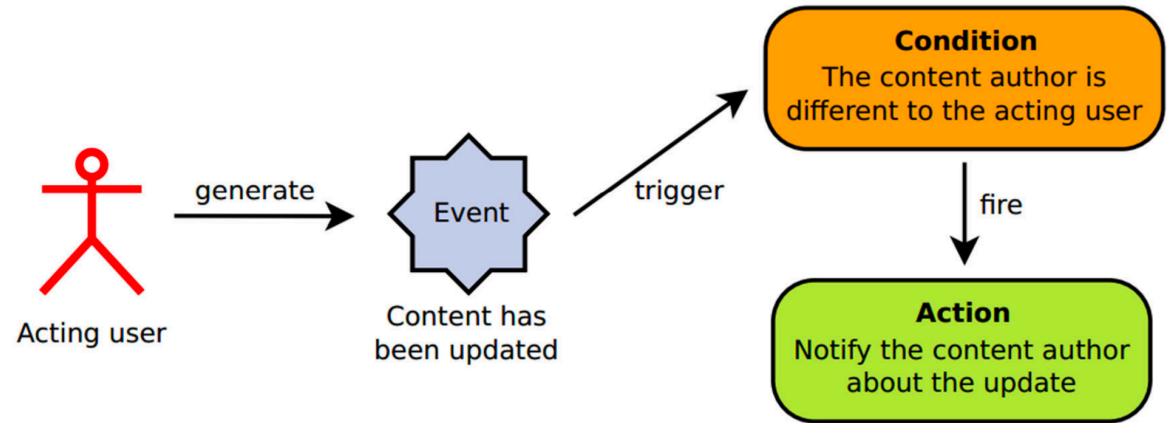
status of order → new →
payed → processing →
shipped

Integrity Constraints

- Assertions have been part of the SQL since SQL-92 (DDL)
- Not supported by most DBMS (e.g., MySQL, Postgres and Oracle)
- If the concept of assertions is to be simulated TRIGGER
- Concept:
 - ▶ Whenever anything is modified in the database, the assertion checks its condition
 - ▶ If the SELECT-statement gives a non-empty result, the operation that has triggered the assertion is denied

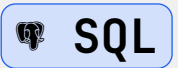
Integrity Constraints - ECA

- ECA rules
 - ▶ on event (E)
 - ▶ under certain conditions (C)
 - ▶ perform actions (A)



Trigger Syntax

```
1  CREATE
2      [DEFINER = user]
3      TRIGGER trigger_name
4      trigger_time trigger_event
5      ON tbl_name FOR EACH ROW
6      [trigger_order]
7      trigger_body
8  trigger_time: { BEFORE | AFTER }
9  trigger_event: { INSERT | UPDATE | DELETE }
10 trigger_order: { FOLLOWS | PRECEDES } other_trigger_name
```



Excursion Delimiter

- A PostgreSQL client program such as pgadmin or psql program uses the delimiter (“;”) to separate statements and executes each statement separately
- However, a stored procedure consists of multiple statements separated by a semicolon (“;”)
- If you use a PostgreSQL client program to define a stored procedure that contains semicolon characters, the PostgreSQL client program will not treat the whole stored procedure as a single statement, but many statements.
- Therefore, you must redefine the delimiter temporarily so that you can pass the whole stored procedure to the server as a single statement.
- To redefine the default delimiter, you use the delimiter command

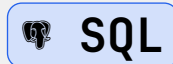
Excursion Delimiter

- In short: A delimiter is a separator between commands



Example

```
1 delimiter |
2 ...
3 |
4 delimiter ;
```



In the code block between “delimiter” and “delimiter;” the delimiter is changed to “|” (instead of “;”)

Excursion Delimiter: Example

```
1  delimiter |
2  CREATE TRIGGER SALARY_VIOLATION
3  BEFORE INSERT ON EMPLOYEE
4  FOR EACH ROW
5  BEGIN
6      IF NEW.SALARY > (SELECT SALARY
7                      FROM EMPLOYEE
8                      WHERE SSN = NEW.SUPER_SSN )
9      THEN SET NEW.Salary = (SELECT SALARY
10                           FROM EMPLOYEE
11                           WHERE SSN = NEW.SUPER_SSN ) - 1;
12  END IF;
13 END;
14 |
15 delimiter;
```

 SQL

Events

- Triggers can react on events
 - ▶ DML: INSERT, UPDATE, DELETE
 - Most common trigger types
 - ▶ DDL: CREATE, ALTER, DROP
 - ▶ DB: startup, shutdown, logon of a user
- No COMMIT triggers

Types

- Time of execution, relative to event
 - ▶ BEFORE
 - ▶ AFTER
- INSTEAD OF
- Statement trigger
 - ▶ Once per statement
 - ▶ Even if no row is affected!
 - ▶ Default trigger type
- Row trigger
- For every affected row

- Syntax: FOR EACH ROW

Order of Trigger

- Before Statement Trigger (once!)
- For every row affected:
 - ▶ Before row trigger
 - ▶ DML operation
 - ▶ Immediate integrity checks
 - ▶ After row trigger
- After Statement Trigger (once!)

Transition Variables

- Row triggers can access old and new tuples
 - ▶ PostgreSQL
 - `old` → NULL for INSERT
 - `new` → NULL for DELETE
 - Oracle
 - ▶ `:new` and `:old`
 - ▶ Before row triggers:
 - Can even modify new!

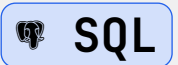
Use Cases

- Constraints on state transitions
- Audit
 - ▶ When was a record last modified?
- Integrity checks with error correction
 - ▶ Change new
- Maintain redundant data
- Updateable views
 - ▶ INSTEAD OF

Trigger: Example

- Audit insertion of new persons

```
1 CREATE OR REPLACE FUNCTION f_trg_emp_log() RETURNS TRIGGER
  $$
2 BEGIN
3   -- Statement here
4 END;
5 $$ LANGUAGE plpgsql;
6
7 DROP TRIGGER IF EXISTS emp_insert;
8 CREATE TRIGGER emp_insert AFTER INSERT ON employee
9 FOR EACH ROW EXECUTE FUNCTION f_trg_emp_log();
```



Trigger: Example

- Salary of new persons

```
1 delimiter |
2 CREATE PROCEDURE output
3     (in ssn char(9), in old_sal DECIMAL(10,2),
4     in new_sal DECIMAL(10,2), in diff_sal DECIMAL(10,2))
5 BEGIN
6     INSERT INTO EMPLOYEE_SALDIFF VALUES ( ssn , old_sal , new_sal,
7     diff_sal);
8 |
9 delimiter;
```

Problems

- Cascading triggers
 - ▶ Trigger actions cause other triggers to fire
- Execution order
 - ▶ Result of high-level operation must be independent hereof!
- “Mutating Tables”

Problems

- Hard to implement
 - ▶ Transaction save!
 - ▶ Multi-session save
- Hard to debug
 - ▶ Update may lead to insert in another table
 - ▶ ... can cause for example constraint violation
 - ▶ Which statement failed?

Assignment: Webshop

- Suppose the following relations in your database
- In the table `Price_History` we want to track on how the prices of the products of table `Product` develop over time. Table `Price_History` has four attributes:
 - ▶ The record ID `PHID`
 - ▶ The reference to table `Product` with the foreign key `PID`
 - ▶ The current price `Price`
 - ▶ The date `Change_Date`, where we store the date of the change

Table `Product`

<u>PID</u>	Price	Description
1	0.50	red apple
2	0.60	green apple
3	1.20	red pepper
4	1.10	green pepper
...	...	...

Table `Product_History`

<u>PHID</u>	PID (FK)	Price	Change_Date
1	1	0.50	02.06.2021
2	3	1.20	02.06.2021
3	2	0.60	03.06.2021
4	4	1.10	04.06.2021
...	...	...	...

Assignment: Webshop

1. INSERT trigger: We want to get an INSERT with the current (start) price in table Price_History when we do an INSERT in the table Product. This is triggered when an INSERT on our table product is done (AFTER).
2. DELETE trigger: Furthermore, in case of a DELETE, all records of the deleted product in the table Price_History should be deleted as well.
3. UPDATE trigger: If a price of a product is changed, this change should also result in an entry in the table Price_History.

Table Product

<u>PID</u>	Price	Description
1	0.50	red apple
2	0.60	green apple
3	1.20	red pepper
4	1.10	green pepper
...	...	...

Table Product_History

<u>PHID</u>	PID (FK)	Price	Change_Date
1	1	0.50	02.06.2021
2	3	1.20	02.06.2021
3	2	0.60	03.06.2021
4	4	1.10	04.06.2021
...	...	...	...

Permissions

- DBMS are multi-user systems
- You need permissions to do anything with the DB:
 - ▶ login
 - ▶ CREATE table, DROP table, etc.
 - ▶ SELECT
 - ▶ INSERT, UPDATE, DELETE
- Permissions can be GRANTED and REVOKED

GRANT and REVOKE

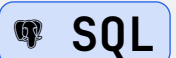
- Permissions can be GRANTED and REVOKED

```
1 GRANT <privilege_name> ON <object_name>  
2 TO { <user_name> | PUBLIC | <role_name>} [ WITH GRANT OPTION ] ;
```



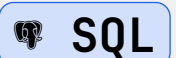
- GRANT

```
1 GRANT SELECT ON tab_a TO user_a;  
2 GRANT UPDATE ON tab_b TO user_a;
```



- REVOKE

```
1 REVOKE SELECT ON tab_a FROM user_a;
```



Least Privilege Principle

- A user should have exactly the permissions necessary to do the work
 - ▶ ... and not more!
- Important for web applications
 - ▶ anonymous end users
 - ▶ not trustworthy
- Limit the possible damage of attacks

Assignment: Webshop

1. Create a user student which is allowed to query and insert the table Product.
2. Revoke the insert privilege from a user student.

4. License Notice

4.1 Attribution

The basis of this course stems from: Professor Dr. Ulrike Herster The following license applies to all unmodified and modified material used throughout this course.

! License

The publication or sharing of slides, image and sound recordings of this course are not permitted.

© Professor Dr. Ulrike Herster

This video and the presented script are protected by copyright. The use is only permitted in relation with the course of study. It is not permitted to forward or republish it in other places (especially in the internet).